

CS101: Computer Programming



C Fundamentals – Part 1

Soumitra Ghosh
Department of Computer Science and
Engineering

soumitra.cse@iitbhu.ac.in

Program: Printing a Pun

```
#include <stdio.h>

int main(void)
{
    printf("To C, or not to C: that is the question.\n");
    return 0;
}
```

- **This is the source code of the program.**
- This program source might be stored in a file named `pun.c`.
- The file name doesn't matter, but the `.c` extension is often required.

Compiling and Linking

- Computer can execute only an executable code.
- The source code of a program can be converted to an executable by the following three steps:
 - **Preprocessing.** The *preprocessor* obeys commands that begin with # (known as *directives*). The preprocessor is usually integrated with the compiler.
 - **Compiling.** A *compiler* translates then translates the program into machine instructions (*object code*).
 - **Linking.** A *linker* combines the object code produced by the compiler with any additional code needed to yield a complete executable program.

Integrated Development Environments

- An *integrated development environment (IDE)* is a software package that makes it possible to edit, compile, link, execute, and debug a program without leaving the environment.
 - Editor of IDE provides easy editing of a source code.
 - Compiler and Linker generates executable code for the source code.
 - Executed code processes data as described in program.
 - Debugger provides tools to execute the code line by line, and to trace the values of the variables in the program.

The General Form of a Simple Program

- Simple C programs have the form
directives

```
int main(void)
{
    statements
}
```

The General Form of a Simple Program

- C uses **curly braces** { and } in much the same way that some other languages use words like `begin` and `end`.
 - They group a set of sentences to a single compound sentence.
- Even the simplest C programs rely on three key language features:
 - Directives
 - Functions
 - Statements

Directives

- Before a C program is compiled, it is first edited by a preprocessor.
- Commands intended for the preprocessor are called directives.
- `#include <stdio.h>`
- `<stdio.h>` is a ***header*** containing information about C's standard I/O library.
- `<stdlib.h>` is a similar ***header*** to include standard library that contains several type conversion and string manipulation functions.

Directives

- Directives always begin with a # character.
- By default, directives are one line long; there's no semicolon or other special marker at the end.

Functions

- A *function* is a series of statements that have been grouped together and given a name.
- *Library functions* are provided as part of the C implementation.
- A function that computes a value uses a `return` statement to specify what value it “returns”:

```
int add_one(int x)
{
    return x + 1;
}
```

The `main` Function

- The `main` function is mandatory.
- `main` is special: it gets called automatically when the program is executed.
- `main` returns a status code; the value 0 indicates normal program termination.
- If there's no `return` statement at the end of the `main` function, many compilers will produce a warning message.

Statements

- A *statement* is a command to be executed when the program runs.
- `pun.c` uses only two kinds of statements. One is the `return` statement; the other is the *function call*.
- Asking a function to perform its assigned task is known as *calling* the function.
- `pun.c` calls `printf` to display a string:

```
printf("To C, or not to C: that is the  
question.\n");
```

Statements

- C requires that each statement end with a semicolon.
 - There's one exception: the compound statement.
- Directives are normally one line long, and they don't end with a semicolon.

Printing Strings

- Characters enclosed in double quotation marks form a string literal.
- When the `printf` function displays a *string literal* it doesn't show the quotation marks.
- `printf` doesn't automatically advance to the next output line when it finishes printing.
- `\n` in a string implies the *new-line character*.
- To make `printf` advance one line, include `\n` in the string to be printed.

Printing Strings

- The statement

```
printf("To C, or not to C: that is the question.\n");
```

could be replaced by two calls of `printf`:

```
printf("To C, or not to C: ");  
printf("that is the question.\n");
```

- The new-line character can appear more than once in a string literal:

```
printf("Brevity is the soul of wit.\n  
--Shakespeare\n");
```

Comments

- A *comment* begins with `/*` and end with `*/`.
- Comments may appear almost anywhere in a program, either on separate lines or on the same lines as other program text.
- Comments may extend over more than one line.

```
/* This is an ANSI-C comment */
```

```
/* Name: pun.c  
   Purpose: Prints a bad pun.  
   Author: K. N. King */
```

Comments

- *Warning:* Forgetting to terminate a comment may cause the compiler to ignore part of your program:

```
printf("My ");      /* forgot to close this comment...  
printf("cat ");  
printf("has ");     /* so it ends here */  
printf("fleas");
```

Comments in C99

- In C99, comments can also be written in the following way:

```
// This is a C99-line comment
```

- This style of comment ends automatically at the end of a line.
- Advantages of `//` comments:
 - Safer: there's no chance that an unterminated comment will accidentally consume part of a program.
 - Multiline comments stand out better.

Variables and Assignment

- Most programs need to a way to store data temporarily during program execution.
 - Data may be of different forms
 - characters. Examples: ‘a’, ‘B’, ‘0’, ‘.’
 - text strings. Examples: “Famagusta”, “Trodos” , “Ali”
 - integers. Examples: 30, -430, 5, -5
 - real numbers. Examples: 2.54. 5.0, 20.35
 - a collection of text, integer and real numbers. Example: “Group1”
- Storage location of these kind data is called *variable*.
 - A variable must have a data format to indicate which type of variable it is.
 - A variable is conveniently called by a *variable-name*.

Types

- Every variable must have a *type* (=format of data).
- C has basic types for numbers
 - `int` (32-bit signed integer from -2,147,483,647 to 2,147,483,647)
 - `float` (32-bit floating point real number, 7-digit)
- and more types
 - `char` (8-bit signed integer for character codes)
 - From -128 to 127, it may code one of 256 characters.
 - `short` (16-bit signed integer from -32535 to 32536)
 - `long` (64-bit signed integer)
 - `double` (64-bit floating point real number, 15-digit)

Types

- A `float` variable can store much larger numbers than an `int` variable.
 - It is short name for *floating-point*
- A `float` variable can store fractional numbers with digits after the decimal point, like 379.125 .

Data types

- A data **type** defines the set of values for a variable
- There are four groups of data types in C:

Basic

Primary data types (integers, characters, etc.)

Enumerated

Integer values associated to labels

Void

No value (for functions without return or generic pointers)

Derived

Arrays, unions, structures, and pointers

Data types - Basic Types

- The basic types in C are the following:

Data type	Description	Typical memory size (in bytes)	Range	Format specifier
char	Characters with sign	1	-128 to 127	%c
unsigned char	Characters without sign	1	0 to 255	%c
short	Short integers with sign	2	-32,768 to 32,767	%hi or %hd
unsigned short	Short integers without sign	2	0 to 65,535	%hu
int	Integers with sign	4	-2,147,483,648 to 2,147,483,647	%i or %d
unsigned int	Integers without sign	4	0 to 4,294,967,295	%u
long	Long integer with sign	8	-9.2e18 to 9.2e18	%li or %ld
unsigned long	Long integer	8	0 to 1.8E19	%lu
float	Decimal	4	1.1e-38 to 3.4e38	%f
double	Decimal with double precision	8	2.2e-308 0 to 1.7e308	%lf

Data types - Basic Types

A character variable holds ASCII value (integer number between 0 and 127)

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

ASCII: Giving Characters a Digital Identity

ASCII — American Standard Code for Information Interchange

Computers store and process numbers. ASCII provides a standard numerical code for representing characters.

Why do we need ASCII?

A computer does not inherently understand:

A B C a b c 0 1 ? @

It works with **bits (0s and 1s)**.

So we need a standard mapping:

Character → Numerical Code → Binary

Character	ASCII (Decimal)	Binary
A	65	1000001
B	66	1000010
a	97	1100001
0	48	0110000
Space	32	0100000

How ASCII Represents Characters

ASCII uses 7 bits

$$2^7 = 128$$

Therefore, standard ASCII provides

128 possible codes:

0000000 $\rightarrow$ 0

:

1111111 $\rightarrow$ 127

These codes represent:

- **Letters** $\rightarrow$ A–Z, a–z
- **Digits** $\rightarrow$ 0–9
- **Punctuation & symbols** $\rightarrow$! @ #
\$ % ...
- **Control characters** $\rightarrow$ Tab, Enter, Escape, etc.

Important distinction

Character	ASCII code
-----------	------------

Binary

'A'	65	1000001
-----	----	---------

'1'	49	0110001
-----	----	---------

'1' is a character, not the numerical value 1.

Similarly:

'A' $\neq$ 1

'A' $\neq$ 65

65 is simply the code assigned to the character A.

ASCII Has Structure!

ASCII codes are not random

- **Uppercase letters**

A = 65

B = 66

C = 67

⋮

Z = 90

- **Lowercase letters**

a = 97

b = 98

c = 99

⋮

z = 122

A useful observation

'a' - 'A' = 32

The difference between uppercase and lowercase letters is **32**.

Why is this useful?

It makes certain character-processing operations simple for computers.

For example:

'A' → 'B' → 'C'

65 → 66 → 67

So the computer can exploit the numerical ordering of ASCII codes.

ASCII: Printable vs Control Characters

The 128 ASCII codes can broadly be divided into:

Category	Examples
Control characters	NUL, TAB, LF, CR, ESC
Printable characters	Letters, digits, punctuation, space

Examples of control characters

TAB → move to the next tab position

LF (Line Feed) → move to the next line

CR (Carriage Return) → return to beginning of line

ESC (Escape) → used for control/escape operations

Not every ASCII code represents something you can see on the screen.

What Happens When You Type "CAT"?

- **Suppose you type: CAT**
 - The computer does **not** store the visual shapes C-A-T.
 - It can represent them using their character codes:

C	A	T
↓	↓	↓
67	65	84
↓	↓	↓
1000011	1000001	1010100

IDEA: *A text string is ultimately represented as a sequence of character codes, which are stored as bits.*

ASCII vs Extended ASCII vs Unicode

- The limitation of standard ASCII
 - ASCII has only 128 codes
 - Enough for basic English, digits and punctuation—but not enough for the world's writing systems.
- For example:
 - বাংলা, हिन्दी, 中文, العربية, 日本語, 😊
 - cannot all be represented using standard 7-bit ASCII.
- So what came next?
 - ASCII → Extended character encodings → Unicode
- **Unicode**
 - Unicode provides a much larger universal character repertoire.

A → Unicode U+0041

अ → Unicode U+0905

ক → Unicode U+0995

😊 → Unicode U+1F60A

ASCII vs Unicode: The Big Picture

Feature	ASCII	Unicode
Main purpose	Character encoding	Universal character repertoire
Standard ASCII codes	128	Much larger repertoire
Basic English	✓	✓
Digits & punctuation	✓	✓
Indian scripts	✗	✓
Chinese/Japanese scripts	✗	✓
Emoji	✗	✓

Data types - Basic Types

- The first argument in function `printf` (i.e., a string) can include *format specifiers* (i.e., subsequences beginning with `%`)

```
#include <stdio.h>

int main() {
    char character = 'c';
    int integer = 255;
    float float_num = 1.2;
    double double_num = 3.1e33;

    printf("This is a character: %c\n", character);
    printf("This is an integer: %d\n", integer);
    printf("This is a float: %f\n", float_num);
    printf("This is a double: %g\n", double_num);
    printf("This is an integer in hexadecimal: %X\n", integer);

    return 0;
}
```

Output

This is a character: c

This is an integer: 255

This is a float: 1.200000

This is a double: 3.1e+33

This is an integer in hexadecimal: FF

Size of Operator

3
0

The **sizeof** operator in C is a compile-time unary operator used to determine the exact **memory size of a data type, variable, or expression in bytes**.

Key Characteristics

- **Compile-Time Execution:** The compiler computes the size before the program runs, meaning it adds zero runtime overhead.
- **No Evaluation:** Expressions inside **sizeof** are not actually executed. For example, **sizeof(x++)** will evaluate the size of **x**, but **x** will not increment.
- **Return Type:** It returns an unsigned integer of type **size_t**.
- **Format Specifier:** You must use the **%zu** format specifier to safely print its value.

Usage of sizeof() Operator

sizeof() operator is used in different ways according to the operand type.

1. When the operand is a Data Type: *sizeof()* simply returns the amount of memory allocated to that data types.

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("The size of a CHAR is %ld bytes\n", sizeof(char));
```

```
    printf("The size of a SHORT is %ld bytes\n", sizeof(short));
```

```
    printf("The size of a INT is %ld bytes\n", sizeof(int));
```

```
    printf("The size of a LONG is %ld bytes\n", sizeof(long));
```

```
    printf("The size of a FLOAT is %ld bytes\n", sizeof(float));
```

```
    printf("The size of a DOUBLE is %ld bytes\n", sizeof(double));
```

```
    printf("The size of a Pointer: %ld bytes\n", sizeof(void *));
```

```
    return 0;
```

```
}
```

Note: *sizeof()* may give different output according to machine. This program is run on a 64-bit gcc compiler.

Output

The size of a CHAR is 1 bytes

The size of a SHORT is 2 bytes

The size of a INT is 4 bytes

The size of a LONG is 4 bytes

The size of a FLOAT is 4 bytes

The size of a DOUBLE is 8 bytes

The size of a Pointer is 8 bytes

Usage of sizeof() Operator

3
0

2. When the operand is an expression: When *sizeof()* is used with the expression, it returns the size of the expression.

```
#include <stdio.h>
int main()
{
    int a = 0;
    double d = 10.21;
    printf("%lu", sizeof(a + d));
    return 0;
}
```

Output

8

sizeof() Operator: Compiler Time Operator

sizeof() is a compile-time operator. compile time refers to the time at which the source code is converted to a binary code. It doesn't execute (run) the code inside ().

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int y;
```

```
    int x = 11;
```

```
    // value of x doesn't change
```

```
    y = sizeof(x++);
```

```
    // prints 4 and 11
```

```
    printf("%i %i", y, x);
```

```
    return (0);
```

```
}
```

Output

4 11

Need of sizeof() Operator

1. To find out the number of elements in an array: Sizeof can be used to calculate the number of elements of the array automatically.

```
#include <stdio.h>
int main()
{
    int arr[] = {1, 2, 3, 4, 7, 98, 0, 12, 35, 99, 14};
    printf("Number of elements: %lu", sizeof(arr) / sizeof(arr[0]));
    return 0;
}
```

Output

Number of elements:11

Usage of sizeof() Operator

2. To allocate a block of memory dynamically: If we want to allocate memory that is sufficient to hold 10 integers and we don't know the sizeof(int) in that particular machine. We can allocate with the help of sizeof.

```
#include <stdlib.h>
int main()
{
    // Allocate memory for 10 integers
    int *arr = (int *) malloc(10 * sizeof(int));
    if (arr == NULL)
        return 1; // Memory allocation failed!
}
```

```
// Initialize and print the array
for (int i = 0; i < 10; i++)
{
    arr[i] = i + 1;
    printf("%d ", arr[i]);
}
// Free the allocated memory
free(arr);
return 0;
```

Output

1 2 3 4 5 6 7 8 9 10

Data types - Enumerated types

- Enumeration are a user defined data types used to assign names to integer constants
 - These names make a program easy to read and maintain
 - The keyword **enum** is used to declare new enumeration types in C

```
#include <stdio.h>

int main() {
    enum days {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
        SATURDAY, SUNDAY
    };

    enum days today = SATURDAY;
    enum days tomorrow = SUNDAY;

    printf("Today is %d\n", today);
    printf("Tomorrow is %d\n", tomorrow);

    if (today == SATURDAY || today == SUNDAY) {
        printf("It's the weekend!\n");
    }

    return 0;
}
```

Output

Today is 5

Tomorrow is 6

It's the weekend!

Constants

- Anywhere in the program we may need some numbers for their literal value.
 - For example: 8 is literally 8. 'a' is literally letter a
- A number written for its literal value is called a constant value.
 - 8 is an integer constant,
 - 7.65 is a float constant,
 - 6.2e-008 is a float constant, $6.2 \times 10^{-8} = 0.000000062$
 - 'A' is a character constant (coded by 65 as a number)
 - "Ali" is a string constant (coded by 65 a string of characters)

Declarations

- Any variable that is planned to be used in main must be declared before all statements:

```
int main(void)
{
    declarations
    statements
}
```

- In C99, declarations don't have to come before statements.
 - But they have to be at the beginning of a compound statement block that begins with "{".
- Variables must be ***declared*** for their types before they are used.
 - Declaring a variable reserves sufficient space for that variable in memory.
- Variables can be declared one at a time or several at the same time:

```
int height;
float profit;
```

```
int height, length, width, volume;
float profit, loss;
```

Assignment

- An int variable is normally assigned a value of type int, and a float variable is normally assigned a value of type float.
- **Mixing types** (such as assigning an int value to a float variable or assigning a float value to an int variable) is possible but **not always safe**.
- You may assign a variable to a value as many times as you need. The **old value of the variable is lost when you assign it to a new value**.
- Once a variable has been assigned a value, it can be **used to help in computing the value of another variable**:

```
height = 8;  
length = 12;  
width = 10;  
volume = height * length * width;  
/* volume is now 960 */
```

- The right side of an assignment can be a formula (or *expression*, in C terminology) involving constants, variables, functions and operators.

Assignment

- A variable can be given a value by means of **assignment**: `height = 8;`
 - *A constant integer 8 is assigned to the variable height.*
 - *The old value in variable height is lost, now it contains 8.*
 - *The value of variable height in any expression stands for 8.*

Before a variable can be assigned a value—or used in any other way—it must first be declared.

- A constant assigned to a float variable usually contains a decimal point:

```
profit = 2150.48;
```

- It's best to append the letter f to a floating-point constant if it is assigned to a float variable:

```
profit = 2150.48f;
```

Failing to include the f may cause a warning from the compiler.

Initialization

- Some variables are automatically set to zero when a program begins to execute.
- A variable that doesn't have a default value and hasn't yet been assigned a value by the program is said to be *uninitialized*.
- Attempting to access the value of an uninitialized variable may yield an unpredictable result.
- With some compilers, worst behavior—even a program crash—may occur.
- The initial value of a variable may be included in its declaration:

```
int height = 8;
```

The value 8 is said to be an *initializer*.

- Any number of variables can be initialized in the same declaration:

```
int height = 8, length = 12, width = 10;
```

- Each variable requires its own initializer.

```
int height, length, width = 10;  
/* initializes only width */
```

Defining Names for Constants

- Using a feature known as *macro definition*, we can name this constant:

```
#define INCHES_PER_POUND 166
```

- When a program is compiled, the preprocessor replaces each macro by the value that it represents.

- During preprocessing, the statement

```
weight = (volume + INCHES_PER_POUND - 1) / INCHES_PER_POUND;
```

will become

```
weight = (volume + 166 - 1) / 166;
```

- The value of a **macro can be an expression**:

```
#define RECIPROCAL_OF_PI (1.0f / 3.14159f)
```

- If it contains operators, the **expression should be enclosed in parentheses**.
- Using **only upper-case letters** in macro names is a common convention.

Program:

Converting from Fahrenheit to Celsius

- The `celsius.c` program prompts the user to enter a Fahrenheit temperature; it then prints the equivalent Celsius temperature.

- Sample program output:

```
Enter Fahrenheit temperature: 212
```

```
Celsius equivalent: 100.0
```

- The program will allow temperatures that aren't integers.

- Defining SCALE_FACTOR to be (5.0f / 9.0f) instead of (5 / 9) is important.
- Note the use of %.1f to display celsius with just one digit after the decimal point.

celsius.c

```
/* Converts a Fahrenheit temperature to Celsius */  
  
#include <stdio.h>  
  
#define FREEZING_PT 32.0f  
#define SCALE_FACTOR (5.0f / 9.0f)  
  
int main(void)  
{  
    float fahrenheit, celsius;  
  
    printf("Enter Fahrenheit temperature: ");  
    scanf("%f", &fahrenheit);  
  
    celsius = (fahrenheit - FREEZING_PT) * SCALE_FACTOR;  
    printf("Celsius equivalent: %.1f\n", celsius);  
  
    return 0;  
}
```

Exercise

- Write a program that converts the length in inches to the length in centimeters (1 inch = 2.54 cm).

Do not forget

- to include `stdio.h`,
- to start the main function,
- to declare two float variables: `inchlength` and `cmlength`,
- to print a string that asks the length in inches,
- to get the entered float number into `inchlength`
- to calculate `cmlength`
- to print `cmlenght`
- to put return statement and close the main function.

Identifiers

- Names for variables, functions, macros, and other entities are called *identifiers*.
- An identifier may contain letters, digits, and underscores, but must begin with a letter or underscore: `times10 get_next_char _done`

It's usually best to avoid identifiers that begin with an underscore.

- Examples of illegal identifiers: `10times get-next-char`
- C is *case-sensitive*: it distinguishes between upper-case and lower-case letters in identifiers.

- For example, the following identifiers are all different:

`job joB jOb jOB Job JoB JOB JOB`

- Many programmers use only lower-case letters in identifiers (other than macros), with underscores inserted for legibility:

`symbol_table current_page name_and_address`

- Other programmers use an upper-case letter to begin each word within an identifier: `symbolTable currentPage nameAndAddress`
- C places no limit on the maximum length of an identifier.

Keywords

- The following *keywords* can't be used as identifiers:

auto	enum	restrict*	unsigned
break	extern	return	void
case	float	short	volatile
char	for	signed	while
const	goto	sizeof	_Bool*
continue	if	static	_Complex*
default	inline*	struct	_Imaginary*
do	int	switch	
double	long	typedef	
else	register	union	

*C99 only

- Keywords (with the exception of `_Bool`, `_Complex`, and `_Imaginary`) must be written using only lower-case letters.
- Names of library functions (e.g., `printf`) are also lower-case.

Layout of a C Program

- A C program is a series of *tokens*.

- Tokens include:
 - Identifiers
 - Keywords
 - Operators
 - Punctuation
 - Constants
 - String literals

- The statement
`printf("Height: %d\n", height);`
consists of seven tokens:

<code>printf</code>	Identifier
<code>(</code>	Punctuation
<code>"Height: %d\n"</code>	String literal
<code>,</code>	Punctuation
<code>height</code>	Identifier
<code>)</code>	Punctuation
<code>;</code>	Punctuation

Layout of a C Program

- The amount of space between tokens usually isn't critical.
- At one extreme, tokens can be crammed together with no space between them, except where this would cause two tokens to merge:

```
/* Converts a Fahrenheit temperature to Celsius */  
#include <stdio.h>  
#define FREEZING_PT 32.0f  
#define SCALE_FACTOR (5.0f/9.0f)  
int main(void){float fahrenheit,celsius;printf(  
"Enter Fahrenheit temperature: ");scanf("%f", &fahrenheit);  
celsius=(fahrenheit-FREEZING_PT)*SCALE_FACTOR;  
printf("Celsius equivalent: %.1f\n", celsius);return 0;}
```

Layout of a C Program

- The whole program can't be put on one line, because each preprocessing directive requires a separate line.
- Compressing programs in this fashion isn't a good idea.
- In fact, adding spaces and blank lines to a program can make it easier to read and understand.
- C allows any amount of space—blanks, tabs, and new-line characters—between tokens.
- Consequences for program layout:
 - *Statements can be divided* over any number of lines.
 - *Space between tokens* (such as before and after each operator, and after each comma) makes it easier for the eye to separate them.
 - *Indentation* can make nesting easier to spot.
 - *Blank lines* can divide a program into logical units.

Layout of a C Program

- Extra spaces can be added between tokens, BUT NOT within a token without changing the meaning of the program or causing an error.
- Writing

```
float fahrenheit, celsius; /** WRONG **/
```

or

```
fl
```

```
oat fahrenheit, celsius; /** WRONG **/
```

produces an error when the program is compiled.

- Putting a space inside a string literal is allowed, although it changes the meaning of the string.
- Putting a new-line character in a string (splitting the string over two lines) is illegal:

```
printf("To C, or not to C:  
that is the question.\n");
```

```
/** WRONG **/
```

WHY? WHY? WHY?

I AM A
KEYWORD

if

I AM AN
IDENTIFIER

printf

WHY?

References

- Slides on C Fundamentals by K N Kings